

[CS230] Optimization Algorithms: Adam Optimizer

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-4. Neural Networks Basics - *Completed*

Chapter 5. Practical Aspects of Deep Learning (Current Unit)

- 5.1-5.5 Regularization & Data Setup - *Completed*
- 5.6 Mini-batch Gradient Descent - *Completed*
- 5.7 Momentum & RMSprop - *Completed*
- **5.8 Adam Optimizer**
 - * The Ultimate Fusion: Momentum + RMSprop
 - * Bias Correction Mechanism
 - * Hyperparameter Standards ($\alpha, \beta_1, \beta_2, \epsilon$)
 - * Implementation from Scratch
- 5.9 Hyperparameter Tuning Strategy - *Upcoming*

지난 시간 복습 및 연결

우리는 지난 두 강의를 통해 '관성'을 이용해 속도를 높이는 **Momentum**과, '보폭'을 조절해 진동을 줄이는 **RMSprop**을 배웠습니다. 그렇다면 자연스러운 질문이 생깁니다. "이 두 가지 장점을 모두 합칠 수는 없을까?" 그 해답이 바로 **Adam (Adaptive Moment Estimation)**입니다. Adam은 현재 딥러닝 학계와 현업에서 '**Default Optimizer(기본 설정)**'로 통합니다. 어떤 옵티마이저를 쓸지 고민될 때, 일단 Adam을 쓰면 80점 이상은 갑니다.

1 Unit Overview

□ 핵심 목표

이 단원은 현대 딥러닝의 표준인 **Adam Optimizer**의 내부 구조를 해부합니다.

- **통합:** Adam이 Momentum의 평균(1차)과 RMSprop의 분산(2차)을 어떻게 결합하는지 수식으로 이해합니다.
- **보정:** 학습 초기에 0으로 쏠리는 현상을 막기 위한 **편향 보정(Bias Correction)**을 익힙니다.
- **표준:** $\beta_1, \beta_2, \epsilon$ 등 하이퍼파라미터의 국룰(Standard Value)을 배웁니다.
- **구현:** Python으로 편향 보정이 포함된 전체 알고리즘을 밑바닥부터 구현합니다.

2 Essential Terminology

기호	표준값	역할
α	튜닝 필요	학습률 (Learning Rate). 가장 중요함.
β_1	0.9	Momentum 계수. (기울기의 지수 평균)
β_2	0.999	RMSprop 계수. (기울기 제곱의 지수 평균)
ϵ	10^{-8}	안정성 상수. 0으로 나누기 방지.

3 Core Concepts: 최강의 융합

3.1 1. The Fusion Algorithm

Adam은 매 스텝(t)마다 다음 4단계를 수행합니다.

1. **Momentum (v):** 속도를 계산합니다. (1차 모멘트)

$$v_t = \beta_1 v_{t-1} + (1 - \beta_1) dW$$

2. **RMSprop (s):** 가속도 제어(마찰력)를 계산합니다. (2차 모멘트)

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) dW^2$$

3. **Bias Correction (핵심):** 초기 0으로 쏠린 값을 보정합니다.

$$v_t^{corr} = \frac{v_t}{1 - \beta_1^t}, \quad s_t^{corr} = \frac{s_t}{1 - \beta_2^t}$$

4. **Update:** 파라미터를 갱신합니다.

$$W = W - \alpha \frac{v_t^{corr}}{\sqrt{s_t^{corr} + \epsilon}}$$

4 Deep Dive: Bias Correction (편향 보정)

”교수님, 왜 굳이 $(1 - \beta^t)$ 로 나눠주나요?” 이것은 Adam의 정교함을 보여주는 대목입니다.

□ 초기값 0의 저주 (수학적 원리)

우리는 $v_0 = 0$ 으로 시작합니다. 첫 번째 스텝($t = 1$)을 봅시다. ($\beta_1 = 0.9$ 가정)

$$v_1 = 0.9 \times 0 + 0.1 \times dW = 0.1dW$$

문제점: 실제 기울기(dW)의 10분의 1(0.1)밖에 반영되지 않습니다. 학습 초반에 거북이처럼 느려집니다.

해결책 (보정):

$$1 - \beta_1^1 = 1 - 0.9 = 0.1$$

$$v_1^{corr} = \frac{v_1}{0.1} = \frac{0.1dW}{0.1} = dW$$

결과: 보정 덕분에 초기에도 기울기를 100% 반영할 수 있습니다. t 가 커지면 $\beta^t \rightarrow 0$ 이 되어, 분모가 1이 되므로 보정 효과는 자연스럽게 사라집니다.

5 Implementation: Adam from Scratch

Adam 구현 시 가장 중요한 것은 현재 반복 횟수 t 를 추적하는 것입니다.

```
1 import numpy as np
2
3 def update_parameters_with_adam(parameters, grads, v, s, t, learning_rate=0.01,
4                                 beta1=0.9, beta2=0.999, epsilon=1e-8):
5     """
6     t: 현재 iteration count 부터(1 시작해야 함!)
7     v, s: 이전 스텝까지 누적된 Momentum, RMSprop 변수
8     """
9     L = len(parameters) // 2
10    v_corrected = {}
11    s_corrected = {}
12
13    for l in range(1, L + 1):
14        # --- 1. Momentum (v) ---
15        v["dW" + str(l)] = beta1 * v["dW" + str(l)] + (1 - beta1) * grads["dW" + str(l)]
16        v["db" + str(l)] = beta1 * v["db" + str(l)] + (1 - beta1) * grads["db" + str(l)]
17
18        # --- 2. Bias Correction (v) ---
19        # 1 - beta^t 로 나눔
20        v_corrected["dW" + str(l)] = v["dW" + str(l)] / (1 - np.power(beta1, t))
21        v_corrected["db" + str(l)] = v["db" + str(l)] / (1 - np.power(beta1, t))
22
23        # --- 3. RMSprop (s) ---
24        # 기울기 제곱(square) 주의!
25        s["dW" + str(l)] = beta2 * s["dW" + str(l)] + (1 - beta2) * np.square(grads["dW" + str(l)])
26        s["db" + str(l)] = beta2 * s["db" + str(l)] + (1 - beta2) * np.square(grads["db" + str(l)])
27
28        # --- 4. Bias Correction (s) ---
29        s_corrected["dW" + str(l)] = s["dW" + str(l)] / (1 - np.power(beta2, t))
30        s_corrected["db" + str(l)] = s["db" + str(l)] / (1 - np.power(beta2, t))
31
32        # --- 5. Update Parameters ---
33        # 분모: sqrt(s_corr) + epsilon
34        parameters["W" + str(l)] -= learning_rate * (v_corrected["dW" + str(l)] / (np.sqrt(s_corrected["dW" + str(l)] + epsilon)))
35        parameters["b" + str(l)] -= learning_rate * (v_corrected["db" + str(l)] / (np.sqrt(s_corrected["db" + str(l)] + epsilon)))
36
37    return parameters, v, s
```

Listing 1: Adam Optimizer Implementation

6 FAQ & Pitfalls

Iteration Count t 주의 (오해 방지 가이드)

함수를 호출할 때 t 는 반드시 1부터 시작해야 합니다. 만약 $t = 0$ 이면 $1 - \beta^0 = 1 - 1 = 0$ 이 되어 **ZeroDivisionError**가 발생합니다.

Q. Adam이 항상 최고인가요?

A. 대부분의 경우(CV, NLP, GAN) 그렇습니다. 하지만 아주 정교한 수렴이 필요할 때(SOTA

논문 등)는 일반 SGD+Momentum이 더 좋은 성능을 낼 때도 있습니다. 그래도 시작은 무조건 Adam을 추천합니다.

다음 단계 (Next Step)

이로써 우리는 최적화 알고리즘의 정점인 Adam을 정복했습니다. 이제 여러분은 어떤 모델이든 빠르고 안정적으로 학습시킬 수 있는 엔진을 갖췄습니다.

하지만 엔진 성능이 좋아도, 기어 변속(하이퍼파라미터 설정)을 잘못하면 차가 나가지 않습니다. α , β , 배치 크기, 은닉층 개수... 도대체 무엇부터 조절해야 할까요? 다음 시간에는 이 수많은 다이얼을 어떤 순서로 돌려야 하는지, [Hyperparameter Tuning]의 체계적인 전략(Random Search vs Grid Search)을 알려드리겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Adam:** Momentum(속도) + RMSprop(가속도 제어) + Bias Correction(초기 보정).
2. **Standard Params:** α (튜닝), $\beta_1(0.9)$, $\beta_2(0.999)$, $\epsilon(10^{-8})$.
3. **Bias Correction:** 학습 초반에 파라미터 업데이트가 너무 작아지는 것을 막아준다.
4. **Memory:** v 와 s 를 따로 저장해야 하므로 일반 SGD보다 메모리를 더 쓴다.